Lightweight SIMT Core Designs for Intelligent 3D Stacked DRAM

Chad D. Kersey
Georgia Institute of Technology
School of ECE
cdkersey@gatech.edu

Hyesoon Kim
Georgia Institute of Technology
College of Computing
hyesoon@cc.gatech.edu

Sudhakar Yalamanchili
Georgia Institute of Technology
School of ECE
sudha@ece.gatech.edu

ABSTRACT

In this work we present an analysis of the Harmonica stream multiprocessor, a light-weight, parameterized, open-source single-instruction-multiple-thread (SIMT) core designed for integration within 3D-stacked DRAM. We evaluate the range of Harmonica designs afforded by the architecture's parameter space in the role of a vault-level accelerator, augmenting a design similar to the Micron Hybrid Memory Cube into an array of compact accelerated DRAM channels. In this role, with a small SRAM cache, Harmonica cores are capable of providing the requisite small footprint, energy efficiency, latency tolerance, and bandwidth demand to perform well. The instruction set and microarchitecture of Harmonica are both novel, providing a lightweight interface for thread creation within the SIMT model and a simple design that issues a single warp per cycle, simplifying the register file design compared to high-performance GPUs, and providing parameters for attributes from the number of warps and threads per warp to the number of general purpose registers per thread. For our suite of analytics-oriented benchmarks, Harmonica cores consuming on the order of 100mW of power maintain a demand for an average of 12GB/s of bandwidth while tolerating the latency present in a DRAM-based memory system.

CCS CONCEPTS

• **Computer systems organization → Single instruction, multiple data**;

KEYWORDS

SIMT, near-memory computing, 3D DRAM, accelerator architectures

ACM Reference format:
Chad D. Kersey, Hyesoon Kim, and Sudhakar Yalamanchili. 2017. Lightweight SIMT Core Designs for Intelligent 3D Stacked DRAM. In *Proceedings of 2017 International Symposium on Memory Systems, Alexandria, VA, October 03–05 2017 (MEMSYS'17),* 11 pages.
https://doi.org/10.1145/3132402.3132426

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

MEMSYS 2017, October 2–5, 2017, Alexandria, VA, USA
© 2017 Association for Computing Machinery.

ACM ISBN 978-1-4503-5335-9/17/10…$15.00
https://doi.org/10.1145/3132402.3132426

1 INTRODUCTION

The computing industry is in the midst of a confluence of technology and application trends due to which the cost, execution time, and energy requirements of applications are being dominated by the memory system. With the end of Dennard scaling, the power per socket will remain roughly flat and therefore sustaining performance scaling will require continuing improvements in energy efficiency. In certain HPC applications the cost of data movement is already exceeding the cost of computation [14]. Further, the growth of signal pins on a socket has not kept pace with the number of cores on each die leading to reduced memory bandwidth per core, further challenging performance scaling. These trends have been exacerbated by the explosive emergence of applications in domains such as machine learning, image processing, graph analytics, and query processing characterized by poor temporal and spatial locality, low operation density, irregular control flow, and irregular memory access patterns. As a result, we see increased stresses on memory bandwidth and latency with negative effects on performance. Consequently, we are in seeing a re-emergence of the need for moving processing to memory to reduce data movement energy, mitigate the impact of poor locality, and increase memory bandwidth available to computations.

While research in the 1990s addressed the idea of placing processing in memory (PIM), the continued evolution of Moores Law and related architectural advances and market forces of the day precluded the need for such architectures. Further, the integration of logic in a DRAM process presented its own challenges. However, with the slowing down of Moore's Law, the emergence of 3D packaging provides a vehicle for integrating logic-optimized and DRAM-optimized dice thereby providing a means to sustain performance scaling by pushing back the memory bandwidth and power walls. However, placement of compute accelerators in memory is subject to distinct constraints and tradeoffs between power, performance, and area. Our goal in the design and implementation our SIMT architecture is to develop an architectural template than can be tuned to match different 3D memory organizations.

In this paper we define a family of lightweight single instruction multiple thread (SIMT) architectures referred to as the Heterogeneous Architecture Research Prototype (HARP). The HARP infrastructure defines a family of instruction set architectures parameterized by aspects such as the word size, the number of general-purpose and predicate registers, and the size and number of synchronously executed thread

blocks. Their simplified design realizes high bandwidth, latency tolerance, low area, and low power. The execution model is a generalization of the bulk synchronous parallel (BSP) model supported by commodity general purpose GPUs and languages such as OpenCL and CUDA. Our refinements to the BSP model add support for dynamic parallelism and exceptions. An implementation of the HARP instruction sets named Harmonica has been produced using an open-source C++-based hardware design library called CHDL.

CHDL is used to produce both a simulation model and a gate-level netlist, using a 15nm standard cell library, for the Harmonica core design, which is configured in terms of these parameters. A system deploying one of these architectures in combination with a stacked DRAM could size the cores so they fit in a tiled multicore arrangement along with per-core caches and independent DRAM channels. In our analysis we run a set of analytics and data warehousing oriented benchmarks. A companion toolchain provides an assembler, linker, emulator and gate and cycle level simulators also using the same set of parameters. We use this environment to generate, explore, and evaluate SIMT accelerators for integration with 3D DRAM memories.

The following section provides some background information and motivation for the HARP design. The remaining sections describe the HARP instruction sets and Harmonica implementationm and the paper concludes with the evaluation of Harmonica running a set of benchmark kernels, and a survey of other current work in the space.

2 BACKGROUND AND OVERVIEW

By permitting some operations to be performed on data in-package, (1) the amount of data movement between the processor chip and the memory can be reduced, (2) energy efficiency improved, (3) use of pin bandwidth optimized, and (4) memory access latency substantially reduced, thereby mitigating the performance effects of poor reference locality. The past impediments to combining DRAM and logic in a single process are now precluded by the emergence of 3D stacked DRAM permitting logic-optimized dice and DRAM optimized dice to be integrated within a single package interconnected by through-silicon vias (TSVs).

This may now be considered to be a mature technology. NAND flash manufacturers have been using 3D die stacking for years and several 3D stacked DRAM technologies are now commercially available. Wide I/O-2 is designed for mobile platforms by stacking conventional DRAM on embedded processors (3D interface) [1] while High Bandwidth Memory (HBM), which incorporates a stack of 4 DRAM dice and a single logic die, is designed for high performance processors [25]. In our work we utilize a third class of 3D memories based on the Hybrid Memory Cube from Micron [10], which is organized into many vertical channels called *vaults*.

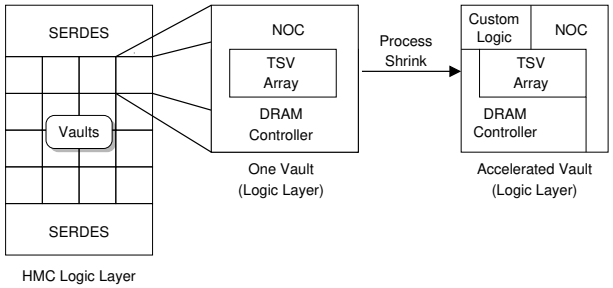

Figure 1: Hypothetical floor-plan of current-generation HMC compared to accelerated vault. At least 1.5mm^2 per vault is expected to be made available for accelerators by moving from a 28nm to a 15nm process.

2.1 Hybrid Memory Cube (HMC)

The Hybrid Memory Cube standard [7] defines an interface and a device architecture for 3D-stacked memory. It is composed of multiple stacked DRAM dice and a single base logic die interconnected with through silicon vias (TSVs). Each DRAM die is divided into partitions in a 2D grid and the corresponding partitions aligned vertically and connected with TSVs form a vertical DRAM channel called a vault. Each vault has an independent DRAM controller on the logic die; therefore each cycle, an access may be initiated in a ready bank in each vault.

These vault controllers are interconnected by an on-chip network consisting of two levels of crossbar switches to high speed external serial links. A typical HMC 2 device provides 8GB of memory organized as 32 vaults, with 4 layers and 4 banks per layer in each vault, or 512 total banks. The logic and memory dice can be fabricated in different process technologies, e.g., DRAM dice are fabricated in 50nm and logic die is fabricated in 28nm [7]. Compared to HBM or Wide I/O-2, the HMC architecture provides highly parallel access to the memory optimized for random accesses; a large number of vaults with one channel per vault and multiple banks per vault, leading to an unprecedented level of parallelism in the memory system. These devices were designed to provide very high bandwidth for random accesses by exploiting this memory-level parallelism and the density of TSVs.

2.2 Architectural Constraints

In a system employing per-vault accelerators, as shown in Figure 1 the accelerator footprint must fit within the vault footprint, including the sizes of the vault controller and router for the on-chip network. If we assume that with the current 28nm process technology, the entire 68mm^2 HMC die area is consumed by the vault memory controller and network interface, then we can estimate that by keeping the DRAM process the same and moving to a more modern 15nm logic process, 1.5mm^2 per vault becomes available for accelerators. Additionally, the proximity of compute and DRAM places tight constraints on the thermal design power (TDP) of

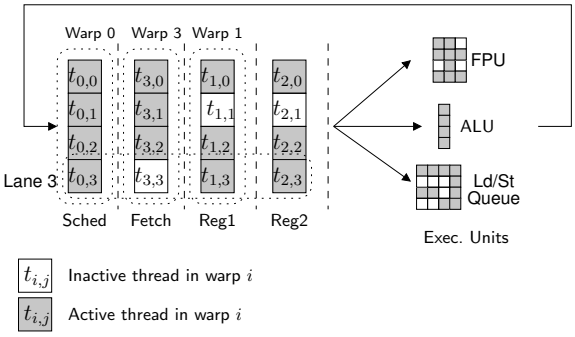

Figure 2: Simplified diagram of an SIMT pipeline.

the logic die, estimated in some studies at 10W [15]. The near-memory accelerators must also be able to effectively utilize the available memory bandwidth especially for those applications plagued by poor spatial and temporal locality.

3 THE HARP ARCHITECTURES

To conform to the power constraints, area constraints and memory bandwidth opportunity within a vault, we selected a single instruction multiple thread (SIMT) architecture, parameterized to fit within the constrained environment of the vault-level accelerator in the logic layer. The parameters that can be adjusted include the size of the general-purpose and predicate register files, the width of the parallel execution units (number of threads per *warp*), and the number of thread blocks or warps. In the following sections we describe a family of such compute architectures, the toolchain for generating architectural instances, and unique features that were adopted to meet the characteristics of emerging applications. Due to the similarities in their SIMT designs, the terminology used in this section is the same terminology used by NVidia when discussing their line of SIMT GPGPUs.

The parameterized family of instruction set architectures (ISAs) we have developed was originally developed as part of a project to create a Heterogeneous Architecture Research Prototype, and is therefore referred to as the family of HARP instruction sets. The HARP ISAs are SIMT architectures that can be described by parameters for the word size, instruction encoding, the number of general-purpose and predicate registers, and the two-dimensional thread count; the number of warps and *lanes*, or threads per warp. Figure 2 illustrates the way in which warps and lanes are grouped logically in a simplified pipelined HARP implementation.

The Heterogeneous Architecture Research Prototype (HARP) defines a space of instruction set architectures that implement a single instruction multiple thread (SIMT) programming and execution model. HARP is composed of two main elements. The first is a parameterized family of instruction set architectures (ISAs) for the SIMT model. An instance of a HARP ISA can be described by parameters for (1) the word size, which must be a multiple of eight bits, (2) instruction encoding, (3) the number of general-purpose and predicate registers, (4) the size of a *warp* which is a synchronously executed block of threads and therefore determines the width of the parallel execution units, and (5) the number of warps supported by an architecture implementation.

3.1 Execution Model

A simplified HARP core pipeline is illustrated in Figure 2. The ISA enables the execution of multithreaded kernels where each kernel is composed of a set of synchronously executed blocks of threads referred to as *warps*. Each warp is composed of a number of threads that execute in lock step and where the number of threads in a warp is equal to number of lanes in an core. In Figure 2, threads are given numbers $t_{i,j}$ where i is the warp ID and j is the position of the thread within the warp. Since kernels executing on SIMT cores must span multiple warps in order to achieve high performance and latency tolerance, a hardware barrier mechanism, described in Section 3.3.3 is provided to synchronize between these warps. The availability of a large number of warps provides for effective latency hiding, while the concurrent execution of threads within warps can generate significant memory traffic that can be satisfied with the high internal memory bandwidth and concurrency within a vault.

3.2 Parameterization

HARP is distinguished by its parameterizability, and implementations of HARP can take on a wide range of performance and energy characteristics, as seen in the evaluation of our implementation called Harmonica in Section 5.

A succinct string representation is used to identify HARP architectures. A `4w32/32/8/4` architecture, for example, has four byte (32-bit) machine words, word-oriented instruction encodings, 32 general-purpose and predicate registers, eight lanes per warp and supports four warps, for a total of 32 hardware threads. Valid HARP architectures have either word-encoded (`w`) or byte-encoded (`b`) instructions and between 1 and 256 predicate registers, registers, warps, and lanes. This format is used by the open-source utility *Harptool*, a monolithic assembler, linker, and emulator utility written in C++, to specify HARP architectures.

3.3 Instruction Set Features

In addition to its parameterization the HARP instruction set provides several unique features in order to implement SIMT semantics and efficiently support application behaviors. These are (1) full predication, (2) `split` and `join` instructions for handling divergent control flow within warps, (3) a barrier instruction and dedicated functional unit, and (4) warp-level exceptions. Some of the more interesting and relevant instructions are listed in Figure 3.

3.3.1 Predication. SIMT cores keep threads within a warp at the same program counter in order to reduce pipeline control overhead. Throughput is maximized by keeping control flow paths in all threads within a warp the same. When the control flow of threads within a warp diverge, such as when a branch in one lane's thread is taken while a branch

Instruction	Operands	Description
`clone %rD`	`%rD`: destination lane	Copy register state to desination thread within warp.
`jalis %rL, %rN, DEST`	`%rL`: Link register. `%rN`: Number of lanes. `DEST`: Destination label.	Jump-and-link-and-spawn; call multithreaded function.
`jmprt %rD`	`%rD`: Destination address.	Jump-and-terminate; return from multithreaded function.
`@pX ? jmpi DEST`	`@pX`: Predicate register. `DEST`: Destination label.	Branches are implemented as predicated jumps.
`split @pX`	`@pX`: Predicate register.	Handle divergent control flow: diverge.
`join`	–	Handle divergent control flow: reconverge.
`bar %rID, %rN`	`%rID`: Barrier ID. `%rN`: # participating warps.	Synchronize warps with barrier.

Figure 3: Selected instructions from the HARP instruction sets.

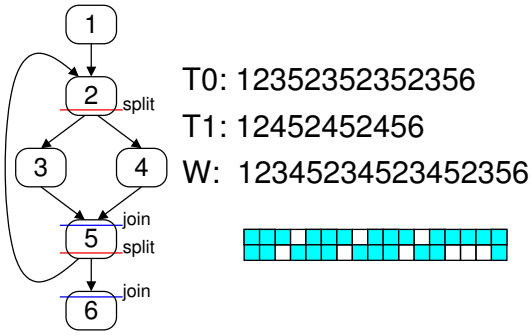

Figure 4: Split and join instructions are used to manage control flow divergence.

in another lane's thread is not taken, a subset of the threads are masked out and not executed; these are shown as the white squares in Figure 2. Two mechanisms are used to serialize the instruction stream between divergent threads. The first, predication, simply uses the value of a 1-bit register to determine whether an instruction will execute. This way, simple `if/then/else` types of statements can be translated to simple assembly code. In HARP, for example, the C-like pseudocode "`if (%r1) %r2++ else %r2--;`" becomes:

```
       rtop @p0, %r1     // reg. to predicate
  @p0 ? addi %r2, %r2, #1 // if @p0, add
       notp @p0, @p0     // complement @p0
  @p0 ? subi %r2, %r2, #1 // if @p0, subtract
```

A stack-based algorithm, discussed in the next section, is used in cases where it is not possible predication e.g. due to looping behavior, or desirable to use predication for performance reasons. All predicated instructions encountered incur the full instruction execution penalty in our implementation, and this is even true when the predicated instruction is not being executed by a single lane.

3.3.2 Split/Join. The immediate post dominator algorithm (IPDOM) first described by NVidia in [18] is a straightforward way to handle the serialization of control flow in cases where predication is not appropriate. IPDOM works by maintaining a per-warp mask of running threads. When a divergent branch occurs, IPDOM pushes a copy of the current thread mask to a special hardware stack, as well as a mask for the not-taken direction of the branch, then sets the mask to contain only the threads that took the branch. At a compiler-selected reconvergence point, which for true IPDOM is the first instruction of the immediate post-dominator of the taken and not-taken paths, the next entry is popped off of the stack and the mask and PC set accordingly.

In the HARP instruction sets, selection of divergence and convergence points is performed explicitly with a pair of instructions. The `split` instruction indicates the point of (possible) divergence, and the `join` instruction indicates the reconvergence point. The example from the previous section, "`if (%r1) %r2++ else %r2--;`", is handled using a divergent branch instead of predication, becoming:

```
        rtop @p0, %r1
  @p0 ? split
  @p0 ? jmp then
        subi %r2, %r2, #1
        jmp cont
else:   addi %r2, %r2, #1
cont:   join
```

when `split` and `join` are used.

3.3.3 Barrier. One of the principles of the bulk synchronous parallel (BSP) execution model used with SIMT processors is that synchronization using barriers is inexpensive. HARP provides inter-warp barrier synchronization through a single instruction. Since threads within warps execute in lockstep, barrier synchronization at every point within a warp is implicit. The barrier instruction, `bar`, takes as operands the number of warps participating in the barrier and the barrier ID. Any number of barrier table entries may be provided by the implementation, allowing for multiple simultaneous barriers to be active. No warp that has executed a `bar` will execute until the number of threads provided in the count operand have also executed `bar` instructions with the same

barrier ID. The way this is implemented in hardware is implementation specific, but in the implementation discussed in the next section, involves table structures containing warp information in a special functional unit.

3.4 Kernel Launch Model

HARP is designed to run independently, minimizing the need for host processor interaction, and as such the HARP instruction sets include support for spawning new warps, and spawning threads within warps. For the benchmarks used in our evaluation, the entire launch and run of the kernel is performed by a single Harmonica core on data that is already present in memory without any interaction with a host processor. Because of these features, HARP can be said to offer a generalization of the BSP execution model and to provide instruction set level hooks for dynamically spawning threads.

When execution on a HARP processor begins, a single warp is running. Executing the `wspawn` instruction spawns a single warp, and copies a given register value into a register of the new warp. This can be used to pass in a pointer to warp parameters and bootstrap the warp spawning process. Within warps, a `clone` instruction is provided to copy register state, and jump-and-link-and-spawn and jump-register-and-terminate instructions are provided to initiate and terminate multi-lane execution, respectively. This allows the parallel portion of execution within a warp to be treated as a simple procedure call.

The `wspawn`, `clone`, and thread control instructions are not priveleged instructions and can be executed by user code, enabling the dynamic parallelism as described in [11] in implementations. These features are made available to the programmer through the HARP runtime library, a lightweight set of functions exposing the ability to spawn warps and call multi-lane functions to the programmer. The purpose of the HARP runtime library is to provide a C API encapsulating the HARP instruction set's thread management features, including control flow divergence and the spawning of threads.

3.5 Exception Support

Rare among SIMT architectures and accelerator-oriented architectures is HARP's ability to handle exceptions through a simple hardware-provided mechanism. Exception handlers in HARP architectures run only in lane 0, and are called in response to events such as page faults and external hardware interrupts. These are implemented in our Harptool emulator but are not supported by the current version of our implementation discussed in the next section, nor are they evaluated in Section 5.

4 THE HARMONICA MICROARCHITECTURE

Harmonica is an implementation of the HARP family of SIMT architectures and ISAs, implemented using CHDL, a hardware description library for C++, designed to enable the creation of hardware using highly templated generators. Harmonica is a small low-power design intended to meet the unique challenges of near-memory computing; a single-issue implementation of a subeset of the HARP parameter space. Using the notation described in Section 3.2, `XwN/N/W/L` architectures are supported by Harmonica, requiring the use of word-oriented encoding, and that the number of general-purpose and predicate registers be the same. This core design achieves the requirements enumerated in Section 2.2 through a variety of techniques:

- Small area and low power through a simple, customizable design that may, when appropriate, eschew energy-and-area demanding featuers such as floating point. Our analysis focuses on fixed point Harmonica cores.
- Tolerance to memory latency through the exploitation of thread-level parallelism. A surplus of warps is maintained that can be executed while other warps await the completion of memory operations.
- The highest possible utilization of memory bandwidth through the use of a multi-lane SIMT design. Each cycle, multiple memory requests can be made simultaneously. Note that parameterization and tool chain makes it possible to generate a single lane Harmonica implementation.

4.1 Pipeline Detail

The design of Harmonica is structured as a modular pipeline, illustrated in Figure 5, in which warps flow through like packets in a network. The semantics of the architecture allow these warps to be completely independent, executing a single instruction on each active lane in each trip through the pipeline.

4.1.1 Scheduler. The scheduler is a simple queue of warp states. Since this is a single issue design, a single warp per cycle proceeds from the scheduler to the fetch stage. Warps are scheduled in a FIFO manner, leading to a simple circular buffer design for the warp table, which dominates the area for this stage but is relatively small given typical numbers of threads per warp.

4.1.2 Instruction Fetch and Register Access. Harmonica uses the CHDL memory interface for both the link to the data cache and instruction cache. This is an arbitrary-width request/response interface with arbitrary word size, which matches requests and responses using tags. Support for atomic operations is provided by the memory interface format through load-linked/store-conditional operations. The memory interface descriptions are provided as part of the CHDL template library making them the standard memory interface used by CHDL designs.

In Harmonica, the warp ID is used as the tag value for both loads and instruction fetches, and the warp state is stored in a table while the operation completes. Warps leave the fetch stage in the order in which their responses return from the instruction cache, and continue to the two register read pipeline stages. Like the scheduler stage, this stage

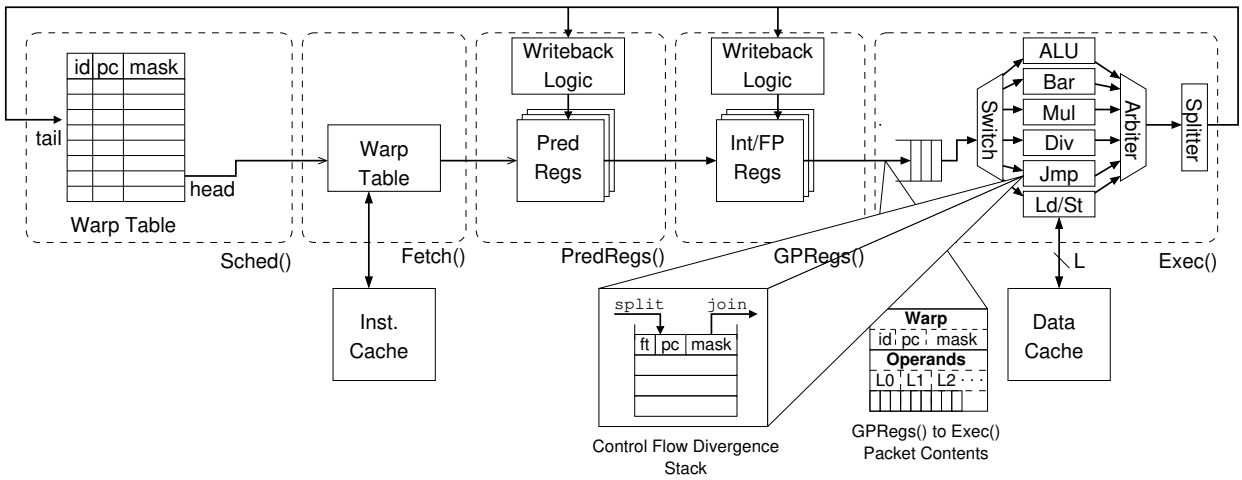

Figure 5: The Harmonica pipeline.

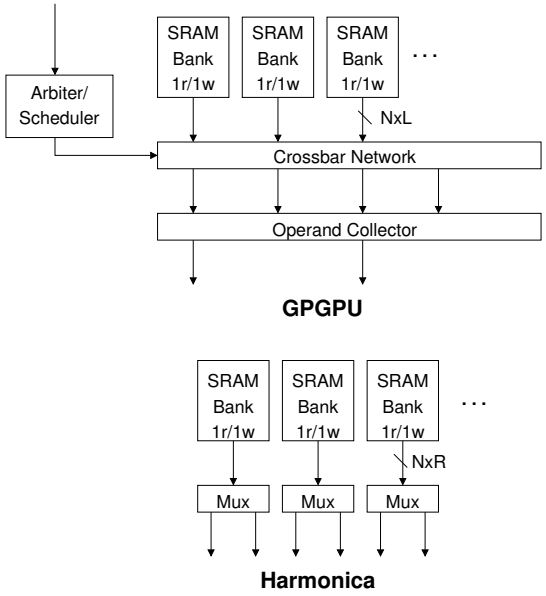

Figure 6: The Harmonica register file design is somewhat simplified compared to that of commercially-available SIMT GPUs.

is dominated in area and gate count by this warp table structure.

4.1.3 Register Files. The use of a single-issue design enables a simplification of the register file, shown in Figure 6, as compared to comparable high-performance GPU designs. Due to the highly-parallel design, typical GPUs require multi-ported register files in order to achieve design performance. Because of the large area penalty of multi-ported memory structures, these are implemented using banking schemes and a large crossbar network. Because the Harmonica design issues only a single warp at a time, its register file design can be greatly simplified, using an SRAM structure with a single read port and a single write port, but foregoing both the bank steering logic and the operand crossbar, saving area and energy at the cost of throughput.

4.1.4 Execute. Once the warps have gathered their operands from the register stages, execution can be performed. Harmonica dispatches warps to functional units by opcode with a simple router. Warps which have finished executing their instructions are returned to the schedule stage, with their program counters and thread masks updated as required by the instruction executed. Concurrently with this their results are written back to the register file.

Because warps share no state, execution units are independent, with an interface consisting entirely of an input and output with a single `ready` bit for backpressure; a FIFO interface. This simplifies the implementation and inclusion of functional units, and as a result, enables customization of Harmonica SMs. For example, all functional units are optional, and some, such as the single-cycle multiply and divide, can be replaced with lower cost bit-serial multi-cycle versions to trade performance, and even, in the case of floating point units, accuracy, for size and power.

Among the functional units present in the execute stage is a simple load/store unit that, much like the fetch unit, sends a request using the CHDL memory request/response interface for each operation, and stores the warp state in a table, with the requirement that there is a memory interface per lane and for loads, all lanes' responses must return before the warp is allowed to advance. This is intended to interface with a small, multi-ported cache with one logical port per SIMT lane.

5 PERFORMANCE ANALYSIS

We have developed a set of open source tools for exploring the HARP design space. Figure 7 shows that the tool

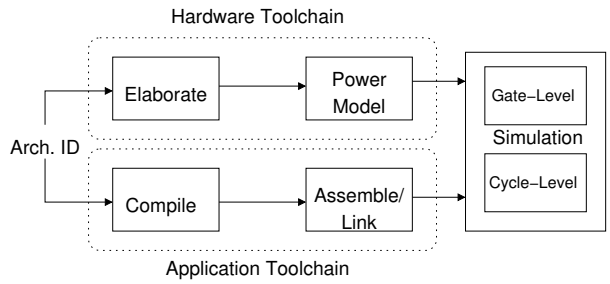

Figure 7: Simplified diagram of tool flow used to evaluate Harmonica design.

flow is bifurcated into two paths; along the bottom is the software path used to compile benchmarks, and along the top is the hardware toolchain used to produce and evaluate Harmonica cores at the gate-level. The software flow includes the benchmarks themselves, their shared runtime library, an LLVM-based compilation infrastructure, and the aforementioned Harptool, which functions both as an assembler/linker program, and as the emulation front-end for our cycle-level simulator. The details of the subset of the hardware simulation flow used to produce each category of results are presented along with those results, but generally a cycle-accurate execute-at-execute Harmonica simulator was produced using the emulator provided in Harptool, in order to quickly produce results. For detailed modeling of power consumption, workloads long enough to get to steady state of the first application phase were run on a gate-level simulation of the Harmonica design synthesized for a 15nm standard cell library instrumented with a counter tree for energy consumption estimation.

Our analysis is performed in the context of a single accelerated vault, considering kernels that operate on data contained entirely within the local vault. While applications operating on larger datasets may experience slow-down due to contention in the on-chip network, there will always be less traffic than the same application would experience running on an off-chip processor. By focusing on the single-vault case, we provide an evaluation of the Harmonica core design disentangled from the orthogonal problems of network-on-chip design and execution models for many-core SIMT processors.

5.1 Benchmarks

Workloads were chosen with an emphasis on memory-bound analytics and database applications that must traverse large in-memory data structures. It was felt both that these kinds of workloads will become increasingly important as the sizes of data sets grow, and that these are the applications for which in-memory processing provides the most benefit. The workloads themselves, listed in Figure 9, are kernels representing common functions that could reasonably be expected to be offloaded by an application to the memory system. These are all fixed-point benchmarks as well. A floating point unit is supported as an optional add-on for Harmonica cores, but is beyond the scope of this work, in which the workloads do not benefit from fast floating point computation.

5.2 Results

The following represents an analysis of the execution of the benchmarks on an Harmonica core that supports integer operations.

5.2.1 Area. A gate-level representation of a range of Harmonica designs has been produced using the technology mapping algorithm provided by CHDL, including a collection of gates from the Open Cell Library for the FreePDK15 15nm PDK [16] and a set of descriptions of SRAM banks. SRAM area was estimated at a bank granularity using a conservative projection of SRAM array area based on values provided by Cacti [26] for the 32nm node, and logic area was estimated as $\frac{\sum_{g \in \text{Gates}} A_g}{0.90}$, the area required for a placement solution with 90% area utilization.

We study a wide range of Harmonica implementations. As seen in Figure 8, many implementations occupy less than a square millimeter; small enough to fit with a 1kB instruction cache and a 32kB data cache in the empty space freed up by shrinking the existing Hybrid Memory Cube logic layer design from 28nm to 15nm. Note that this assessment does not assume any available area in the current HMC logic layer design, a conservative assumption since the HMC vault pitch is likely determined by the DRAM banks, the current generation may have free area in the logic layer already. There is likely area to accommodate multiple Harmonica cores or larger Harmonica cores in a vault.

5.2.2 Power. As a complement to CHDL's technology mapping functionality, a power emulation library has been built that allows the generation, as CHDL hardware, of activity-based energy models. Using the extracted capacitances, and internal switching energies provided along with the Open Cell Library, a set of netlists augmented with energy models was created that could be run through a gate-level simulator. These were simulated at the gate level for long enough to reach steady-state in our set of applications. This produced a set of logic power traces in terms of energy per cycle and SRAM access counts. These were multiplied by Cacti-derived access energy estimates and added to the logic power to produce the final results, which are expressed in milliwatts, assuming a 650MHz clock rate, which is according to [7] the rate at which at least some logic in the HMC operates.

Figure 10 shows the breakdown for power consumption for 1ms of operation at steady state for each of our six applications. Even the largest cores simulated, 16 lanes wide with 16 warps, operate well within the 300mW per core (1 core for each of 32 vaults, approximately 10 watts total power) limit. Of note is the fact that one of the applications, breadth-first search, consistently expends less power than the remaining five. This is because the irregular nature of breadth-first search leads it to have higher than average

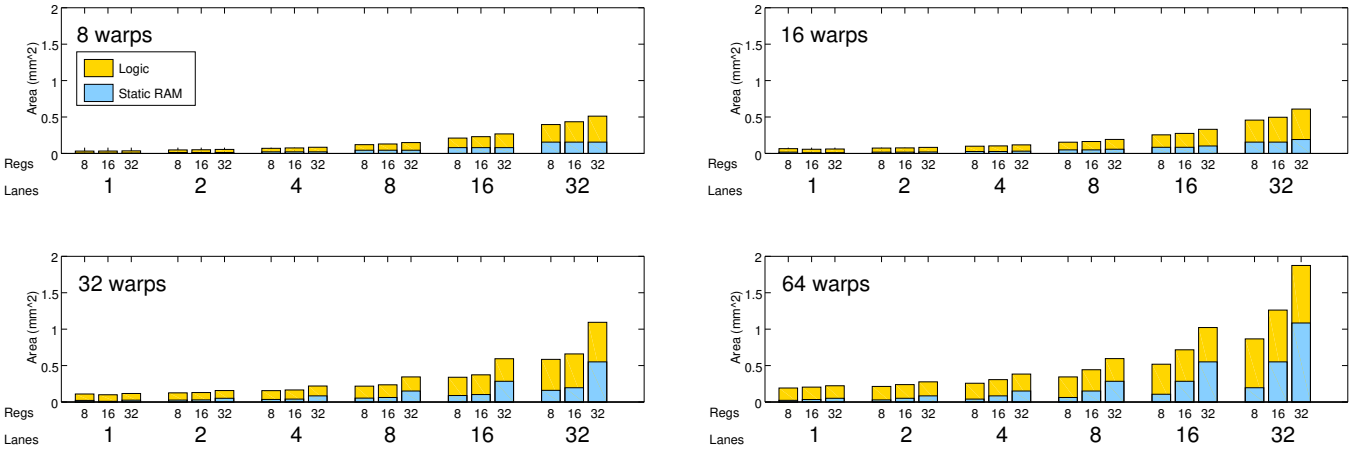

Figure 8: Area estimates for Harmonica cores, not including L1 caches, based on synthesis.

Name	Description	Data	Size
`bfs`	Graph breadth-first search. [17]	Pennsylvania road network.	1090920 nodes, 3083796 edges
`radix`	Radix sort. [23]	Random 32-bit integers.	1048576 elements
`binsearch`	Binary search.	Random 32-bit integers.	1048576 elements, 1048576 lookups
`hashtable`	Hash table lookup.	Random 32-bit integers.	1048576 elements, 1048576 lookups
`vecsum`	Sum integer vector.	Random 32-bit integers.	16777216 elements
`select`	Select from table.	Random booleans.	1048576 elements, 1037940 matching rows

Figure 9: The benchmarks used in our evaluation of the Harmonica core design. Cited papers are sources for algoritms used.

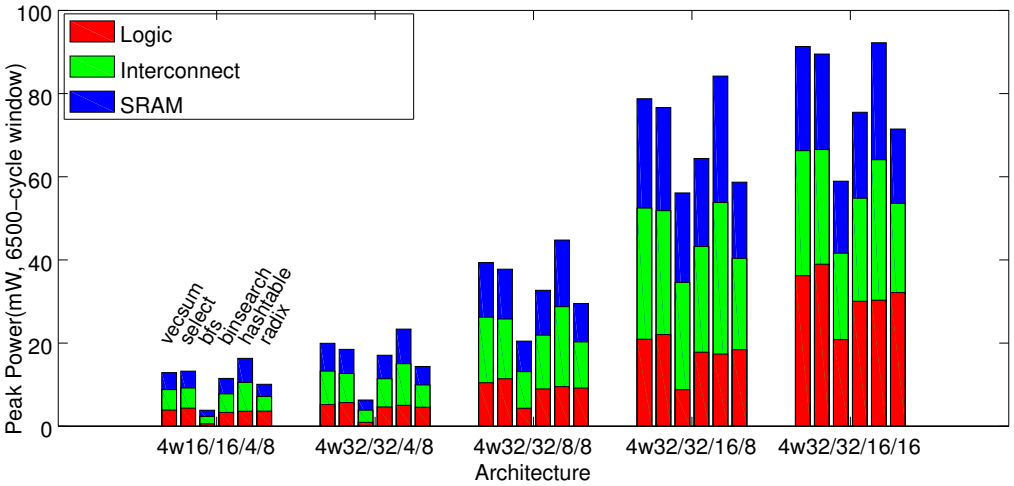

Figure 10: Average power for 1ms of each application running at 650MHz.

control flow divergence and warp load imbalance, limiting the average number of active SIMT lanes and pipeline stages.

5.2.3 Latency Tolerance. To measure latency tolerance, we used a cycle accurate simulator built around the Harptool emulation library. This includes a highly-banked 32kB 4-way set associative cache for data and a 32kB 4-way set associative cache for instructions. For these results, no attempt was made to optimize the data cache address mapping to improve hit rates, and hit rates were typically quite low in the D-cache. Meanwhile, all kernels fit entirely into the I-cache.

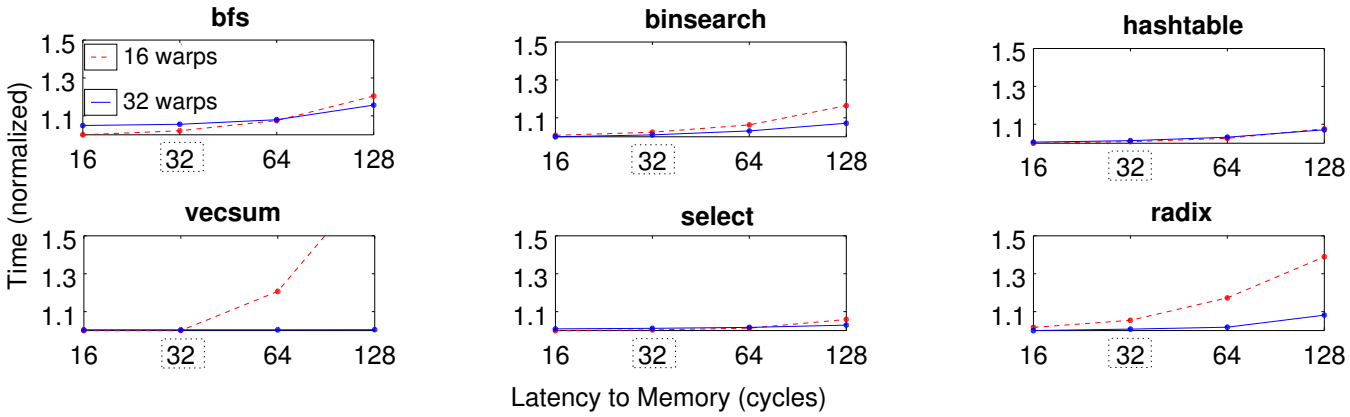

Figure 11: Relative application performance in simulation as a function of memory access latency for `4w32/32/16` architectures with 16 or 32 warps. 32 cycles highlighted as it represents the most realistic latency estimate for local vault accesses.

The DRAM timing model used to produce the latency tolerance results, summarized in Figure 11, is a simple fixed-latency model, since the purpose is observing limits on application performance caused only by limits in latency, without consideration for bandwidth or the address dependence of latency. Bandwidth is given a complete treatment in Section 5.2.4 and the impact of vault bank conflicts and cacheing is examined in Section 5.3 The cores studied all had 32 general-purpose registers per thread and 32 SIMT execution lanes, though the values of these had little impact on relative performance.

Latency tolerance in SIMT cores in general and Harmonica in particular is achieved by having a large number of available warps that can be scheduled, so that a large number of memory operations can be in flight simultaneously, taking advantage of high-bandwidth, high-latency memory like the DRAM used in our HMC-based vaults. The number of available warps is the most obvious factor impacting a Harmonica core's tolerance to memory latency, and as Figure 11 shows, increasing the number of warps effectively increases the tolerance to memory latency. For the applications studied, even the 16-warp Harmonica core is capable of handling the estimated 32-cycle latency of the HMC vault DRAM with negligible performance degradation, and a 32-warp version is capable of handling four times that latency.

5.2.4 Bandwidth Demand. Figure 12 shows the bandwidth achieved by the Harmoncia core for various design sizes on our applications. For this simulation, in order to evaluate the peak bandwidth demand, the latency of a DRAM access was fixed at 32 cycles and the bandwidth was left unbounded, with no bank or channel model in place. In this simulation, the Harmonica core with 32 lanes and 32 warps running at 650MHz was able to achieve quite high utilization of bandwidth; greater than two thirds of the DRAM vault's available 10GB/s on 4 of the 6 applications. The remaining two, hashtable and binsearch, despite their relatively random access patterns to their data stores, use quite a few thread-local variables and thus achieve surprisingly high cache hit rates, leading to lower-than-expected bandwidth utilization.

5.3 Impact of Cache

Given the small size of the benchmark kernel code, less than 5kB on average and 10kB maximum (for breadth-first search), it is unsurprising that the only instruction cache misses in our evaluation are compulsory misses; i.e. the kernels all fit in our 32kB I-cache. While more complex applications may necessitate future exploration of instruction cache designs for SIMT cores, the looping nature of most programs and the fact that each thread in a low-divergence application only performs $\frac{1}{W}$, where W is the number of warps per thread, instruction cache accesses per instruction, de-emphasizes instruction caches in SIMT accelerator design.

Despite the cores' tolerance to memory latency, our simulation results show that the performance of Harmonica cores *is* dependent on the presence and performance of the data cache. This effect is entirely due to the cache's ability to reduce DRAM queuing latency caused by bank conflicts. Figure 13 shows, for a 32-GPR, 16-lane, 32-warp Harmonica core, the impact of cache miss rate on application performance. The cache sweep was performed using an accurate vault performance model, thereby emphasizing the effects of bank conflicts and TSV bandwidth limits. For this simulation, a realistic model of the memory channel was used that, unlike the model used in the latency tolerance study, simulated the closed-page policy of the HMC vault, bank conflicts, and associated queuing delay.

6 RELATED WORK

The use of multithreading for memory latency tolerance and simplified datapath control can be traced back to pioneering "barrel processors" such as the Tera MTA. This technique was notably adopted in commercial SIMT architectures first

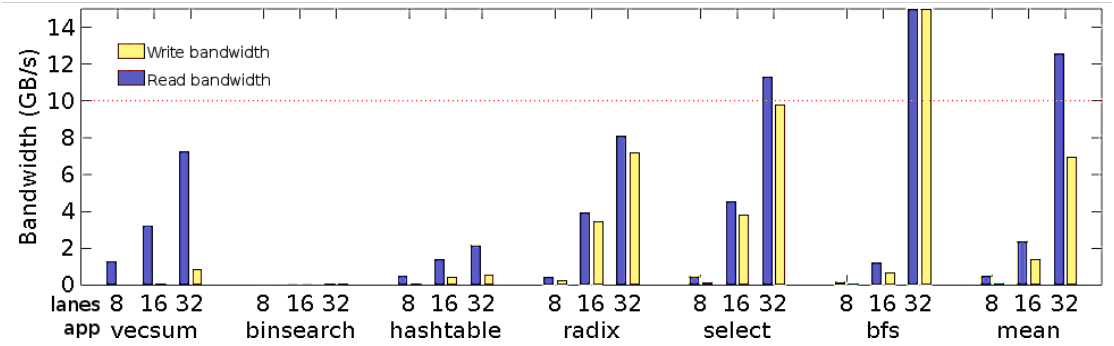

Figure 12: Bandwidth as a function of application and Harmonica core size. The number of warps and the number of lanes are the same in this case, and the number of GPRs is held at 32.

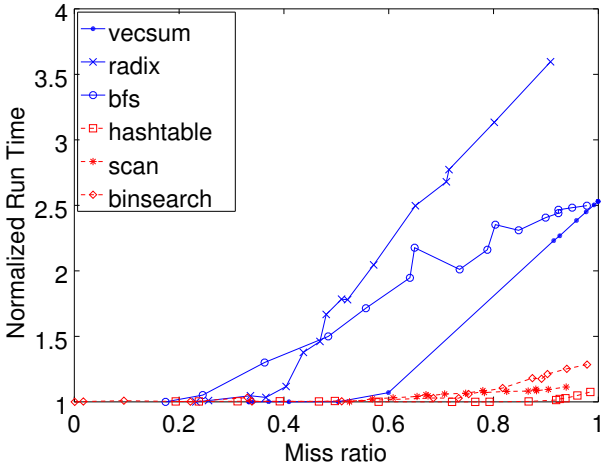

Figure 13: Plot of miss ratio vs. slowdown for various data cache associativities and capacities from 64 bytes to 16 kilobytes, simulated with a realistic vault model.

described in documentation from NVidia alongside a thread-based vector processing approach [18].

An interest in processing in memory in the 1990s led to work including [9] and [19], which ultimately never led to the production of a PIM product. The development in 3D die stacking has led to renewed interest, and near-memory accelerators for a diverse range of applications including general-purpose computing [27], graph processing [2] [28], sparse matrix operations [22], and machine learning [12]. SIMT and other GPU-like processors have been discussed in the near-memory role, including [27], [20], and [21].

Several open source GPUs have been developed for various applications, including FlexGrip [3] and Nyuzi [5]. These processors are similar to Harmonica in that they are, to some extent, parameterizable, but this parameterization does not extend to instruction set aspects such as the number of general-purpose registers. These designs are not specifically targeted toward in-memory applications, and have not been evaluated in detail for physical or performance characteristics that would qualify them as suitable for the study of PIM applications.

Our work with the open source CHDL hardware description environment parallels recent work on the Chisel open source hardware description language and framework [4]. Much like CHDL, Chisel provides an environment for writing generators for hardware using a general-purpose programming language; in the case of Chisel this language is Scala instead of C++. There is quite a bit of work now on accelerator architectures in general, and fixed-function accelerators in particular have received newfound attention as a response to near-memory computing and dark silicon, with tools like Aladdin [24] providing ways to quickly analyze sets of accelerators without performing, as we have for Harmonica, low-level hardware design. Reconfigurable accelerators have also made a comeback in the domain of near-memory processing, with architectures like HRL [8] providing reconfigurable fabrics in the logic layer. Accelerators can be produced for these by a variety of methods, including automatic generation from software source code as demonstrated with the Delite HDL [13]. HARP is described as a customizable architecture in the sense that it is parametric. While this is true, it differs significantly from the kind of low-level architecture generation that is performed in work such as [6].

7 CONCLUSION

A Harmonica core with 32 lanes and 32 warps fits in the logic layer of a Hybrid Memory Cube style vault, can nearly saturate the available bandwidth for many applications while tolerating the latency of directly accessing DRAM, and fits within the 10 watt thermally-imposed power constraint of such near-memory cores.

ACKNOWLEDGMENTS

This research was supported by National Science Foundation under grant CCF-1533767.

REFERENCES

[1] 2014. Wide i/o 2 (wideio2)(jesd229-2). (2014).

[2] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. 2015. A scalable processing-in-memory accelerator for parallel graph processing. In *Computer Architecture (ISCA), 2015 ACM/IEEE 42nd Annual International Symposium on*. IEEE, 105–117.

[3] Kevin Andryc, Murtaza Merchant, and Russell Tessier. 2013. FlexGrip: A soft GPGPU for FPGAs. In *Field-Programmable Technology (FPT), 2013 International Conference on*. IEEE, 230–237.

[4] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avižienis, John Wawrzynek, and Krste Asanović. 2012. Chisel: constructing hardware in a scala embedded language. In *Proceedings of the 49th Annual Design Automation Conference*. ACM, 1216–1225.

[5] Jeff Bush, Mohammad A Khasawneh, Khaled Z Mahmoud, and Timothy N Miller. 2016. NyuziRaster: Optimizing rasterizer performance and energy in the Nyuzi open source GPU. In *Performance Analysis of Systems and Software (ISPASS), 2016 IEEE International Symposium on*. IEEE, 204–213.

[6] Nathan Clark, Jason Blome, Michael Chu, Scott Mahlke, Stuart Biles, and Krisztian Flautner. 2005. An architecture framework for transparent instruction set customization in embedded processors. In *Computer Architecture, 2005. ISCA'05. Proceedings. 32nd International Symposium on*. IEEE, 272–283.

[7] Hybrid Memory Cube Consortium et al. 2013. Hybrid Memory Cube Specification 1.0. (2013).

[8] Mingyu Gao and Christos Kozyrakis. 2016. HRL: efficient and flexible reconfigurable logic for near-data processing. In *High Performance Computer Architecture (HPCA), 2016 IEEE International Symposium on*. Ieee, 126–137.

[9] Maya Gokhale, Bill Holmes, and Ken Iobst. 1995. Processing in memory: The Terasys massively parallel PIM array. *Computer* 28, 4 (1995), 23–31.

[10] Joe Jeddeloh and Brent Keeth. 2012. Hybrid memory cube new DRAM architecture increases density and performance. In *2012 Symposium on VLSI Technology (VLSIT)*.

[11] Stephen Jones. 2012. Introduction to dynamic parallelism. In *GPU Technology Conference Presentation S*, Vol. 338.

[12] Duckhwan Kim, Jaeha Kung, Sek Chai, Sudhakar Yalamanchili, and Saibal Mukhopadhyay. 2016. Neurocube: A programmable digital neuromorphic architecture with high-density 3D memory. In *Computer Architecture (ISCA), 2016 ACM/IEEE 43rd Annual International Symposium on*. IEEE, 380–392.

[13] David Koeplinger, Christina Delimitrou, Raghu Prabhakar, Christos Kozyrakis, Yaqi Zhang, and Kunle Olukotun. 2016. Automatic generation of efficient accelerators for reconfigurable hardware. In *Proceedings of the 43rd International Symposium on Computer Architecture*. IEEE Press, 115–127.

[14] Peter Kogge, Keren Bergman, Shekhar Borkar, Dan Campbell, W Carson, William Dally, Monty Denneau, Paul Franzon, William Harrod, Kerry Hill, et al. 2008. Exascale computing study: Technology challenges in achieving exascale systems. (2008).

[15] Gabriel H Loh. 2012. Computer architecture for die stacking. In *VLSI Design, Automation, and Test (VLSI-DAT), 2012 International Symposium on*. IEEE, 1–2.

[16] Mayler Martins, Jody Maick Matos, Renato P. Ribas, André Reis, Guilherme Schlinker, Lucio Rech, and Jens Michelsen. 2015. Open Cell Library in 15Nm FreePDK Technology. In *Proceedings of the 2015 Symposium on International Symposium on Physical Design (ISPD '15)*. ACM, New York, NY, USA, 171–178. https://doi.org/10.1145/2717764.2717783

[17] Duane Merrill, Michael Garland, and Andrew Grimshaw. 2012. Scalable GPU Graph Traversal. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '12)*. ACM, New York, NY, USA, 117–128. https://doi.org/10.1145/2145816.2145832

[18] S. Moy and J.E. Lindholm. 2005. Method and system for programmable pipelined graphics processing with branching instructions. (Sept. 20 2005). http://www.google.com/patents/US6947047 US Patent 6,947,047.

[19] I Off. 1992. Computational RAM: A memory-SIMD hybrid and its application to DSP. (1992).

[20] Reena Panda, Yasuko Eckert, Nuwan Jayasena, Onur Kayiran, Michael Boyer, and Lizy Kurian John. 2016. Prefetching Techniques for Near-memory Throughput Processors. In *Proceedings of the 2016 International Conference on Supercomputing*. ACM, 40.

[21] Ashutosh Pattnaik, Xulong Tang, Adwait Jog, Onur Kayiran, Asit K Mishra, Mahmut T Kandemir, Onur Mutlu, and Chita R Das. 2016. Scheduling techniques for GPU architectures with processing-in-memory capabilities. In *Parallel Architecture and Compilation Techniques (PACT), 2016 International Conference on*. IEEE, 31–44.

[22] Fazle Sadi, Larry Pileggi, and Franz Franchetti. 2016. 3D DRAM based application specific hardware accelerator for SpMV. In *High Performance Extreme Computing Conference (HPEC), 2016 IEEE*. IEEE, 1–1.

[23] Nadathur Satish, Changkyu Kim, Jatin Chhugani, Anthony D Nguyen, Victor W Lee, Daehyun Kim, and Pradeep Dubey. 2010. Fast sort on CPUs and GPUs: a case for bandwidth oblivious SIMD sort. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*. ACM, 351–362.

[24] Yakun Sophia Shao, Brandon Reagen, Gu-Yeon Wei, and David Brooks. 2014. Aladdin: A pre-rtl, power-performance accelerator simulator enabling large design space exploration of customized architectures. In *Computer Architecture (ISCA), 2014 ACM/IEEE 41st International Symposium on*. IEEE, 97–108.

[25] JEDEC Standard. 2013. High bandwidth memory (hbm) dram. *JESD235* (2013).

[26] Shyamkumar Thoziyoor, Naveen Muralimanohar, Jung Ho Ahn, and Norman P Jouppi. 2008. *CACTI 5.1*. Technical Report. Technical Report HPL-2008-20, HP Labs.

[27] Dongping Zhang, Nuwan Jayasena, Alexander Lyashevsky, Joseph L. Greathouse, Lifan Xu, and Michael Ignatowski. 2014. TOP-PIM: Throughput-oriented Programmable Processing in Memory. In *Proceedings of the 23rd International Symposium on High-performance Parallel and Distributed Computing (HPDC '14)*. ACM, New York, NY, USA, 85–98. https://doi.org/10.1145/2600212.2600213

[28] Jialiang Zhang, Soroosh Khoram, and Jing Li. 2017. Boosting the Performance of FPGA-based Graph Processor Using Hybrid Memory Cube: A Case for Breadth First Search. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '17)*. ACM, New York, NY, USA, 207–216. https://doi.org/10.1145/3020078.3021737